HCLTech - 100 Technical Interview Questions

HCL Technologies

With Answers & Diagrams - For BTech Computer Science / IT Students

Object-Oriented Programming

Q1. Explain the SOLID principles of object-oriented design.

Answer:

SOLID stands for: Single Responsibility (a class should have one reason to change), Open/Closed (open for extension, closed for modification), Liskov Substitution (subtypes must be substitutable for their base types without breaking correctness), Interface Segregation (prefer many small specific interfaces over one large one), and Dependency Inversion (depend on abstractions, not concrete implementations).

Q2. What is constructor overloading? Can constructors be inherited?

Answer:

Constructor overloading means a class has multiple constructors with different parameter lists, letting objects be created in different ways (e.g. Point() vs Point(x,y)). Constructors are NOT inherited by subclasses - a subclass must define its own constructors, though it can call a parent constructor explicitly via super().

Q3. Explain the Open/Closed Principle with a practical example.

Answer:

The Open/Closed Principle says classes should be open for extension but closed for modification. Practical example: instead of adding an if-else chain to a PaymentProcessor class every time a new payment method is added, define a PaymentMethod interface and add new payment types as new classes implementing it - existing code doesn't need to change.

Q4. What is the difference between method overloading and method overriding? Give a code example of each.

Answer:

Overloading means multiple methods with the same name but different parameter lists in the same class (resolved at compile time), e.g. add(int,int) and add(double,double). Overriding means a subclass provides its own implementation of a method already defined in its parent with the same signature (resolved at runtime), e.g. Dog overriding Animal's makeSound().

Q5. Explain what a static method is and why it cannot be overridden (only hidden) in Java.

Answer:

A static method belongs to the class itself rather than any instance, and is resolved at compile time based on the reference type, not the runtime object type. Since overriding relies on runtime (dynamic) dispatch, a static method with the same signature in a subclass merely 'hides' the parent's version rather than overriding it - the version called depends on which class name/reference type you use to call it.

Q6. What are access modifiers? Explain public, private, protected, and default with examples.

Answer:

Access modifiers control visibility: 'public' - accessible from anywhere; 'private' - accessible only within the same class; 'protected' - accessible within the same package and by subclasses (even in different packages, in Java); 'default' (no modifier) - accessible only within the same package. They enforce encapsulation by controlling what other code can touch.

Q7. Explain encapsulation and why it is important for maintainable software.

Answer:

Encapsulation hides an object's internal state behind private fields and exposes controlled access through public getters/setters or methods. It matters because it lets you change internal implementation without breaking external code, enforce validation (e.g. reject a negative balance), and reduces unintended coupling between modules.

Q8. What is the Single Responsibility Principle, and why does violating it make code harder to maintain?

Answer:

The Single Responsibility Principle states a class should have only one reason to change - i.e., it should do one thing well. Violating it (e.g. a class that both parses files and sends emails) means unrelated changes end up modifying the same class, increasing the risk of breaking unrelated functionality and making the class harder to test and reuse.

Q9. What is the difference between composition and inheritance? Which is generally preferred and why?

Answer:

Composition means a class holds a reference to another class as a field ('has-a', e.g. a Car has an Engine), while inheritance means a class extends another ('is-a', e.g. a Car is-a Vehicle). Composition is generally preferred because it's more flexible (behaviour can be swapped at runtime) and avoids the tight coupling and fragile-base-class problems that deep inheritance hierarchies create - this is the idea behind 'favor composition over inheritance'.

Q10. What is object cloning, and what is the difference between a shallow copy and a deep copy?

Answer:

Object cloning creates a copy of an existing object. A shallow copy duplicates the object's top-level fields but nested objects/references are shared with the original (mutating a nested object affects both copies). A deep copy recursively duplicates nested objects too, so the copy is fully independent of the original.

Q11. Explain the concept of a virtual function in C++ and how it enables runtime polymorphism.

Answer:

A virtual function in C++ is declared with the 'virtual' keyword in a base class, telling the compiler to resolve the call at runtime via a vtable based on the actual object type rather than the reference/pointer type. This is what allows a Shape* pointing to a Circle to correctly call Circle's draw() instead of Shape's - the mechanism behind runtime polymorphism in C++.

Q12. Explain the difference between an abstract class and an interface. When would you choose one over the other?

Answer:

An abstract class can mix abstract and concrete methods, hold state (fields), and a class can extend only one abstract class. An interface (pre-Java 8) only declares method signatures with no implementation, and a class can implement many interfaces. Use an abstract class when subclasses share common state/behaviour; use an interface when you only need to guarantee a contract across unrelated classes.

Q13. What is polymorphism? Differentiate between compile-time and runtime polymorphism with examples.

Answer:

Polymorphism lets the same interface behave differently depending on the actual object. Compile-time (static) polymorphism is resolved during compilation - method/operator overloading is the classic example. Runtime (dynamic) polymorphism is resolved during execution based on the actual object type - method overriding via a base-class reference is the classic example.

Q14. What design pattern would you use to ensure only a single instance of a class exists, and how would you implement it thread-safely?

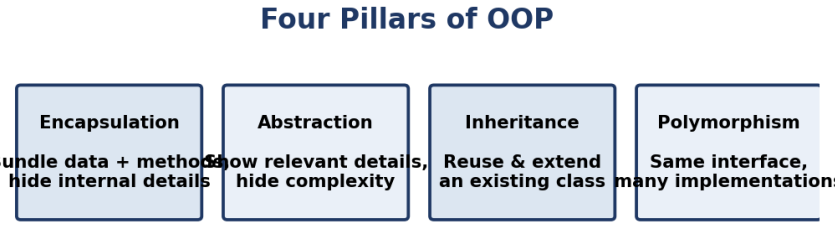
Answer:

The Singleton design pattern ensures only one instance of a class exists. A thread-safe implementation typically uses either double-checked locking with a volatile instance field, an eagerly-initialized static final instance, or (in Java) a single-element enum, all of which avoid two threads creating two separate instances under concurrent access.

Q15. Explain the four pillars of Object-Oriented Programming with a real-world example for each.

Answer:

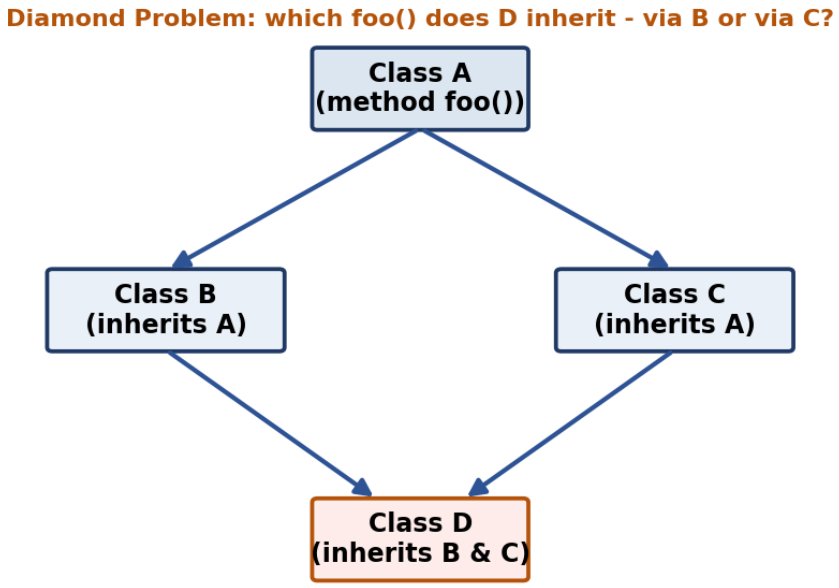
The four pillars are: **Encapsulation** - bundling data and methods together and hiding internal state (e.g. a bank account class exposing deposit()/withdraw() but hiding the raw balance field); **Abstraction** - exposing only relevant details, like a car's steering wheel hiding the mechanics underneath; **Inheritance** - a Manager class reusing and extending an Employee class; **Polymorphism** - a draw() method behaving differently for a Circle vs a Square object called through a common Shape reference.



Q16. What is the 'diamond problem' in multiple inheritance, and how does Java avoid it?

Answer:

The diamond problem arises when a class inherits from two classes that both inherit from a common base class, creating ambiguity about which inherited method version to use. Java avoids it for classes by disallowing multiple inheritance of classes entirely (a class can extend only one class), and for interfaces (Java 8+ default methods) it forces the implementing class to explicitly override the conflicting method if two interfaces provide the same default.



DBMS & SQL

Q17. Explain optimistic vs pessimistic locking in database transactions.

Answer:

Pessimistic locking acquires a lock on a row before reading/modifying it, blocking other transactions until it's released - safe under high contention but can hurt throughput. Optimistic locking assumes conflicts are rare: it reads data without locking, then checks (e.g. via a version number) at commit time whether the data changed since it was read, retrying if a conflict is detected - better throughput when conflicts are infrequent.

Q18. What is the difference between OLTP and OLAP systems?

Answer:

OLTP (Online Transaction Processing) systems handle many short, frequent read/write transactions (e.g. an ATM system) and are optimized for fast inserts/updates on normalized schemas. OLAP (Online Analytical Processing) systems handle complex, read-heavy analytical queries over large historical datasets (e.g. a sales dashboard) and are optimized with denormalized/star schemas for fast aggregation.

Q19. Explain the difference between horizontal and vertical partitioning of a table.

Answer:

Horizontal partitioning splits a table by rows (e.g. orders from 2023 in one partition, 2024 in another), similar in spirit to sharding but often within a single database instance. Vertical partitioning splits a table by columns (e.g. splitting rarely used large text/blob columns into a separate table), reducing the size of rows scanned for common queries that don't need those columns.

Q20. Explain the ACID properties of a database transaction with an example for each.

Answer:

ACID: **Atomicity** - a transaction's operations either all succeed or all fail together (e.g. a bank transfer's debit and credit both happen or neither does); **Consistency** - a transaction moves the database from one valid state to another, respecting constraints; **Isolation** - concurrent transactions don't see each other's incomplete/intermediate changes; **Durability** - once committed, changes survive a crash (written to persistent storage).

Q21. How would you optimize a slow-running SQL query? Walk through your debugging approach.

Answer:

Approach: first run EXPLAIN/EXPLAIN ANALYZE to see the query plan and spot full table scans; check whether the filtered/joined columns have appropriate indexes; look for functions applied to indexed columns (which can prevent index use); check for unnecessary SELECT * pulling extra columns; consider rewriting correlated subqueries as joins; and check table statistics are up to date so the optimizer picks a good plan.

Q22. Explain the difference between a clustered and a non-clustered index.

Answer:

A clustered index determines the physical storage order of the table's data rows - a table can have only one, since data can only be sorted one way physically. A non-clustered index is a separate structure that stores pointers back to the actual data rows, so a table can have several, useful for speeding up queries on columns other than the clustering key.

Q23. What is the difference between DELETE, DROP, and TRUNCATE?

Answer:

DELETE removes rows one at a time (can be filtered with WHERE, is logged, and can be rolled back) and fires triggers; DROP removes the entire table structure and its data permanently; TRUNCATE quickly removes all rows by deallocating data pages (minimal logging), resets identity counters, but keeps the table structure - and cannot be rolled back in most databases once committed.

Q24. Explain the difference between a foreign key constraint and a check constraint.

Answer:

A foreign key constraint enforces referential integrity - it ensures a column's value matches an existing value in the referenced table's primary/unique key (or is NULL), preventing orphaned references. A check constraint enforces a custom condition on the values allowed in a column (e.g. age >= 18), independent of any other table.

Q25. What is a trigger, and give an example of when you would use one.

Answer:

A trigger is a stored procedure automatically executed in response to an INSERT, UPDATE, or DELETE event on a table. Example: a trigger that automatically logs the old and new values of a Salary column into an AuditLog table whenever an Employee row is updated, without requiring the application code to do it explicitly.

Q26. What is database replication, and what is the difference between synchronous and asynchronous replication?

Answer:

Database replication maintains copies of data across multiple servers for redundancy and read scaling. Synchronous replication waits for the replica to confirm the write before the transaction commits (stronger consistency, higher write latency); asynchronous replication commits on the primary immediately and propagates changes to replicas afterward (lower latency, but a small risk of data loss if the primary fails before replicating).

Q27. Explain referential integrity and what happens when you try to delete a row referenced by a foreign key.

Answer:

Referential integrity ensures a foreign key value always points to an existing row in the referenced table (or is NULL). Trying to delete a row that is still referenced by a foreign key will fail with a constraint violation error, unless the foreign key was defined with ON DELETE CASCADE (deletes dependent rows too) or ON DELETE SET NULL (nulls out the reference) behavior.

Q28. Write a SQL query to find employees who earn more than the average salary of their department.

Answer:

SELECT e.name, e.salary, e.department FROM Employee e WHERE e.salary > (SELECT AVG(salary) FROM Employee e2 WHERE e2.department = e.department). The correlated subquery recomputes the department's average salary for each row and compares it to that employee's own salary.

Q29. How does a B-Tree index improve query performance internally?

Answer:

A B-Tree index keeps keys sorted in a balanced tree structure with high fan-out, so the database can locate a row's location in only a handful of disk-page reads (typically O(log n)) instead of scanning every row, by repeatedly narrowing the search range as it walks down the tree from the root to a leaf pointing at the actual row.

Q30. What are ACID vs BASE properties, and where would you prefer a BASE-compliant (NoSQL) system?

Answer:

ACID (Atomicity, Consistency, Isolation, Durability) prioritizes strict correctness and is the model used by traditional relational databases. BASE (Basically Available, Soft state, Eventual consistency) relaxes strict consistency in favor of availability and partition tolerance, common in NoSQL systems. You'd prefer a BASE system for massive-scale, highly available workloads (e.g. a social media feed) where brief inconsistency is acceptable in exchange for uptime and horizontal scalability.

Q31. Write a SQL query to find the Nth highest salary using a window function like RANK() or DENSE_RANK().

Answer:

SELECT name, salary FROM (SELECT name, salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM Employee) ranked WHERE rnk = N (substituting the desired N). DENSE_RANK() assigns ranks without gaps even when there are salary ties, making it more robust than ROW_NUMBER() for this use case.

Q32. Write a SQL query to find duplicate records in a table.

Answer:

Using GROUP BY with HAVING COUNT(*) > 1: SELECT column_name, COUNT(*) FROM table_name GROUP BY column_name HAVING COUNT(*) > 1. This groups rows by the column(s) that define a duplicate and returns only the groups appearing more than once.

Q33. Write a SQL query to find the second-highest salary from an Employee table without using LIMIT/TOP.

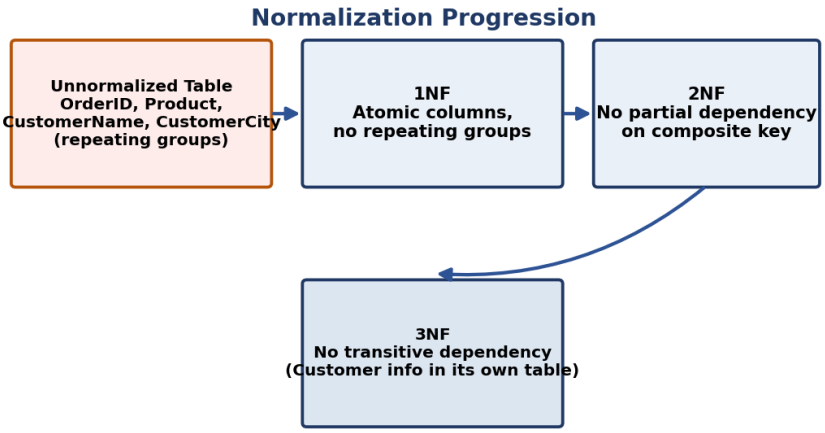
Answer:

Using a subquery: SELECT MAX(salary) FROM Employee WHERE salary < (SELECT MAX(salary) FROM Employee). This first finds the overall maximum, then finds the maximum salary that is strictly less than it, giving the second-highest value without relying on database-specific LIMIT/TOP syntax.

Q34. What is normalization? Explain 1NF, 2NF, 3NF, and BCNF with a practical example.

Answer:

Normalization removes redundancy step by step: **1NF** requires atomic column values with no repeating groups; **2NF** requires 1NF plus no partial dependency of a non-key column on part of a composite primary key; **3NF** requires 2NF plus no transitive dependency (a non-key column depending on another non-key column); **BCNF** is a stricter version of 3NF handling certain anomalies where a table has multiple overlapping candidate keys. Example: splitting an Orders table that repeats CustomerCity for every order into a separate Customers table removes the transitive dependency.

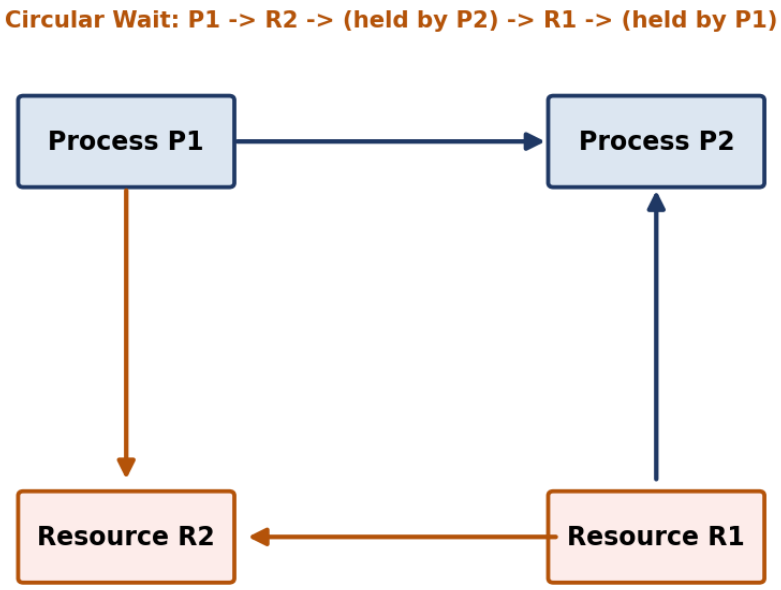


Operating Systems

Q35. Explain the four necessary conditions for a deadlock to occur.

Answer:

The four necessary conditions are: **Mutual Exclusion** - at least one resource is held in a non-shareable mode; **Hold and Wait** - a process holds one resource while waiting for another; **No Preemption** - resources can't be forcibly taken away, only released voluntarily; **Circular Wait** - a closed chain of processes each waiting for a resource held by the next. All four must hold simultaneously for a deadlock to occur.



Q36. What is a race condition, and how can synchronization primitives like semaphores or mutexes prevent it?

Answer:

A race condition occurs when multiple threads/processes access shared data concurrently and the final result depends on the unpredictable timing/order of execution, potentially producing incorrect results. Synchronization primitives like mutexes (mutual exclusion locks) or semaphores prevent this by ensuring only one thread can enter a critical section that modifies shared data at a time.

Q37. What is thrashing, and what causes it?

Answer:

Thrashing happens when a system is so overcommitted on memory that it spends most of its time swapping pages in and out of disk rather than executing actual process instructions, causing CPU utilization and throughput to collapse even though the system appears busy. It's typically caused by too many processes running with too little physical memory, so each one's working set doesn't fit in RAM.

Q38. How does an operating system ensure fair CPU allocation among multiple processes?

Answer:

The OS ensures fair CPU allocation through its scheduler, using algorithms like Round Robin (equal time slices for each process) or priority-based scheduling with aging (gradually increasing the priority of processes that have waited a long time, to prevent starvation), combined with preemption so no single process can monopolize the CPU indefinitely.

Q39. Explain the producer-consumer problem and how you would solve it using semaphores.

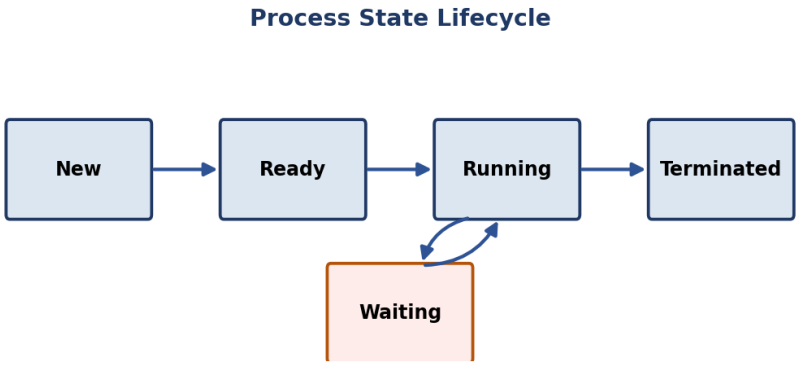
Answer:

The producer-consumer problem involves a producer adding items to a shared bounded buffer and a consumer removing them, needing synchronization so the producer doesn't add to a full buffer and the consumer doesn't remove from an empty one. It's solved using two counting semaphores (one tracking empty slots, one tracking filled slots) plus a mutex protecting the buffer itself during actual insert/remove operations.

Q40. What is the difference between a process and a thread?

Answer:

A process is an independent program in execution with its own memory space (code, data, heap, stack) and OS-allocated resources. A thread is a lightweight unit of execution within a process that shares the process's memory and resources with other threads of the same process, making thread creation and context switching cheaper than process creation.



Q41. What is the Banker's Algorithm, and how does it help avoid deadlocks?

Answer:

The Banker's Algorithm avoids deadlock by only granting a resource request if the resulting state is 'safe' - meaning there exists some order in which all processes could still finish given the maximum resources they might request. It simulates allocation before actually granting it, refusing requests that could lead to an unsafe (potentially deadlocked) state.

Q42. Explain the difference between preemptive and non-preemptive scheduling with examples of each.

Answer:

Preemptive scheduling allows the OS to forcibly suspend a running process to give the CPU to another (e.g. Round Robin, Priority Scheduling with preemption) - responsive but has context-switch overhead. Non-preemptive scheduling lets a running process keep the CPU until it voluntarily yields or finishes (e.g. FCFS, non-preemptive SJF) - simpler but a long process can block shorter ones ('convoy effect').

Q43. What is virtual memory, and why is it useful even when physical RAM is limited?

Answer:

Virtual memory gives each process the illusion of a large, contiguous address space by mapping logical addresses to physical RAM or disk (swap space) via page tables, transparently to the program. It's useful even with limited RAM because it allows programs larger than physical memory to run, isolates each process's memory from others for safety, and lets the OS overcommit memory across many processes that don't all need their full memory at once.

Q44. Describe a real-world analogy to explain deadlock to a non-technical person.

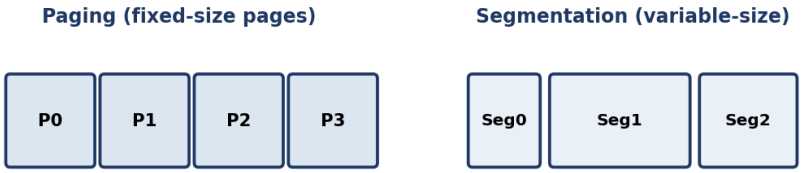
Answer:

Analogy: imagine four cars at a four-way intersection, each waiting for the car to its right to move first before it can proceed - if all four wait simultaneously, none will ever move, even though each car is only blocked because of the others' identical behavior. That's a deadlock: everyone is waiting for someone else to act first, and no one ever will.

Q45. Explain the difference between paging and segmentation as memory management techniques.

Answer:

Paging divides both physical and logical memory into fixed-size blocks (pages/frames), eliminating external fragmentation but potentially causing internal fragmentation within a page. Segmentation divides a program into variable-sized logical units (code, stack, heap segments) that map more naturally to how programmers think about memory, but can suffer external fragmentation since segments vary in size.



Q46. Compare FCFS, SJF, Round Robin, and Priority scheduling algorithms - what are the trade-offs of each?

Answer:

FCFS (First-Come-First-Served) is simple but can cause the 'convoy effect' where a long job delays all others. SJF (Shortest Job First) minimizes average waiting time but requires knowing job lengths in advance and can starve long jobs. Round Robin gives each process a fixed time quantum, ensuring fairness and responsiveness for interactive systems, but too small a quantum increases context-switch overhead. Priority Scheduling runs the highest-priority job first but can starve low-priority jobs unless aging is used.

Computer Networks

Q47. What is a VPN, and how does it provide secure remote access over a public network?

Answer:

A VPN (Virtual Private Network) creates an encrypted tunnel between a client and a remote network over a public network like the internet, so traffic appears to originate from the remote network and can't be read by anyone intercepting it in transit. This lets remote employees securely access internal company resources as if they were on the local network.

Q48. Explain what a firewall does and the difference between a stateful and stateless firewall.

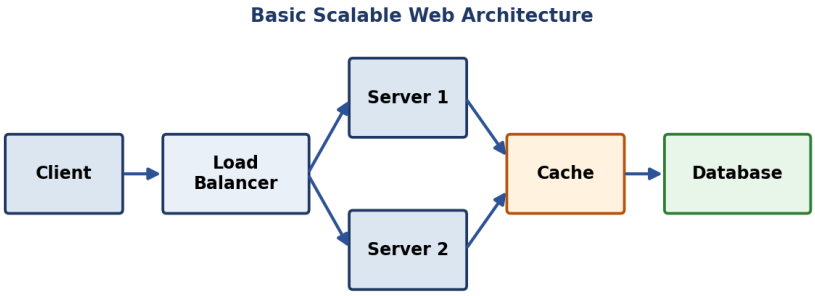
Answer:

A firewall monitors and controls incoming/outgoing network traffic based on defined security rules. A stateless firewall evaluates each packet independently against static rules (e.g. block port 23), without knowledge of ongoing connections. A stateful firewall tracks the state of active connections and can make smarter decisions, like automatically allowing return traffic for a connection that was initiated from inside the network.

Q49. Explain the concept of load balancing and name a few common load-balancing algorithms.

Answer:

Load balancing distributes incoming requests across multiple backend servers to avoid overloading any single one and to improve availability. Common algorithms: Round Robin (cycles through servers in order), Least Connections (sends traffic to the server with the fewest active connections), and IP Hash (routes a given client consistently to the same server based on a hash of its IP, useful for session persistence).



Q50. What is ARP, and how does it map an IP address to a MAC address?

Answer:

ARP (Address Resolution Protocol) resolves a known IP address to its corresponding MAC address on a local network. When a device needs to send a frame to an IP it doesn't have a MAC address for, it broadcasts an ARP request ('who has this IP?'), and the device owning that IP responds with its MAC address, which the sender then caches for future use.

Q51. What is the difference between HTTP and HTTPS, and how does TLS/SSL secure a connection?

Answer:

HTTP sends data in plaintext, so anyone intercepting the traffic can read it. HTTPS wraps HTTP inside a TLS/SSL layer that encrypts the data in transit. Under the hood, a TLS handshake occurs first: the client and server agree on a cipher suite, the server presents a certificate to prove its identity, and both sides derive a shared symmetric session key used to encrypt all subsequent HTTP traffic.

Q52. Explain the difference between IPv4 and IPv6.

Answer:

IPv4 uses 32-bit addresses (about 4.3 billion possible addresses, written as four decimal octets like 192.168.1.1) and is running out of available addresses. IPv6 uses 128-bit addresses (an astronomically larger space, written in hexadecimal groups like 2001:0db8::1), and also simplifies header processing and has built-in support for features IPv4 needed extensions for, like auto-configuration.

Q53. What is a CDN, and how does it reduce latency for end users?

Answer:

A CDN (Content Delivery Network) is a geographically distributed network of cache servers that store copies of static content (images, videos, scripts) closer to end users. When a user requests content, it's served from the nearest CDN edge server rather than the origin server, reducing round-trip latency and offloading traffic from the origin.

Q54. What is NAT, and why is it necessary given the limited IPv4 address space?

Answer:

NAT (Network Address Translation) allows multiple devices on a private network to share a single public IP address when communicating with the internet, translating private IPs to the public IP (and back for return traffic) at the router/gateway. It's necessary because IPv4's roughly 4.3 billion addresses are far fewer than the number of internet-connected devices worldwide.

Q55. Explain the difference between unicast, multicast, and broadcast communication.

Answer:

Unicast sends data from one sender to exactly one specific receiver (e.g. a normal web request). Multicast sends data from one sender to a specific group of interested receivers simultaneously (e.g. IPTV streaming to subscribed clients). Broadcast sends data from one sender to every device on the local network segment, regardless of interest (e.g. ARP requests, DHCP discovery).

Q56. What is a subnet mask, and how do you calculate the number of usable hosts in a subnet?

Answer:

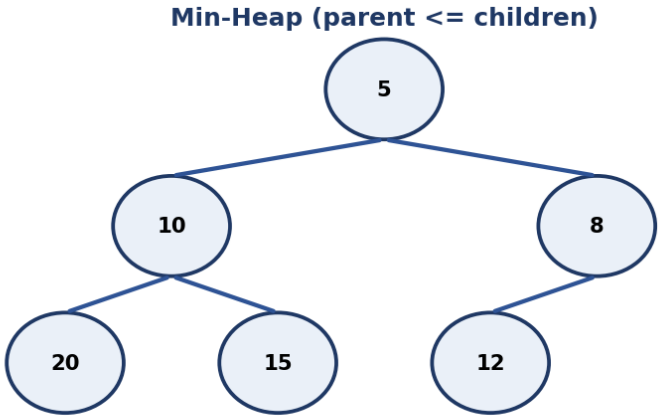
A subnet mask defines which portion of an IP address is the network part and which is the host part (e.g. 255.255.255.0 means the first 24 bits are network, the last 8 are host). For a /24 subnet, there are 2^8 = 256 total addresses, minus 2 reserved (network address and broadcast address), giving 254 usable host addresses.

Data Structures & Algorithms

Q57. Explain how a min-heap/max-heap is implemented and how it supports efficient priority queue operations.

Answer:

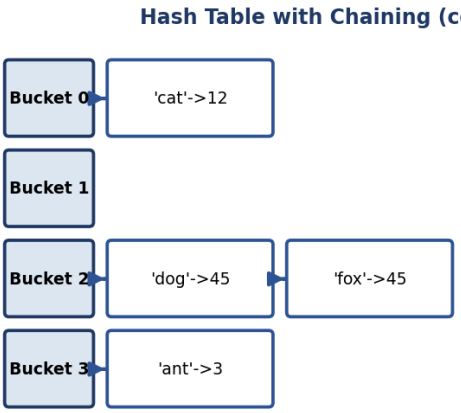
A heap is a complete binary tree (usually implemented as an array) satisfying the heap property: in a min-heap, every parent is less than or equal to its children (so the minimum is always at the root); in a max-heap, every parent is greater than or equal to its children. Insertion and extraction of the min/max both take O(log n) since you only need to 'bubble' an element up or down the height of the tree, making heaps the standard backing structure for an efficient priority queue.



Q58. Explain how a hash table works internally, including collision resolution strategies.

Answer:

A hash table maps keys to array indices using a hash function, giving average O(1) time for insert/search/delete. Since different keys can hash to the same index (a collision), collisions are typically resolved via chaining (each bucket holds a linked list/array of entries that hashed there) or open addressing (probing for the next free slot in the array itself, e.g. linear or quadratic probing).



Q59. What is the sliding window technique, and in what type of problems is it useful?

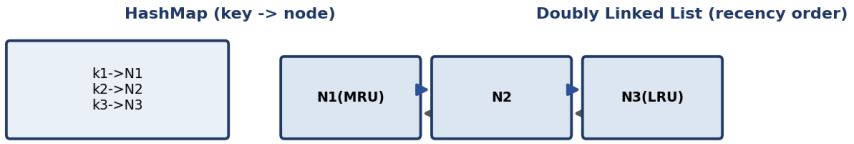
Answer:

The sliding window technique maintains a contiguous subrange ('window') over an array or string and expands/shrinks its boundaries incrementally rather than recomputing from scratch for every possible subrange. It's especially useful for problems asking for the best/longest/shortest subarray or substring satisfying some condition (e.g. maximum sum subarray of size k, or longest substring without repeating characters), typically turning an O(n^2) brute force into an O(n) solution.

Q60. Explain how you would design an LRU cache and the data structures you'd use to achieve O(1) get/put operations.

Answer:

An LRU cache needs O(1) get and put operations. The standard design combines a hashmap (key -> pointer to a node) for instant lookup with a doubly linked list ordered by recency of use: on access, the accessed node is unlinked and moved to the 'most recently used' end in O(1); when the cache exceeds capacity, the node at the 'least recently used' end is evicted, also in O(1), with the hashmap updated accordingly.

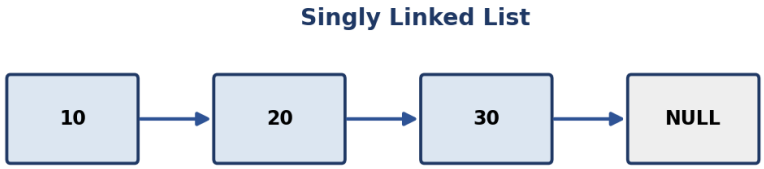


O(1) get/put: hashmap finds the node instantly, doubly linked list reorders it to MRU end

Q61. How would you detect a cycle in a linked list? Explain Floyd's cycle detection algorithm.

Answer:

Floyd's cycle detection (the 'tortoise and hare' algorithm) uses two pointers moving through the linked list at different speeds - one advances one node at a time (slow), the other advances two nodes at a time (fast). If there is a cycle, the fast pointer will eventually 'lap' the slow pointer and they will meet at the same node; if the fast pointer reaches the end (null), there is no cycle. This runs in O(n) time and O(1) space.



Q62. Explain the difference between time complexity and space complexity with examples.

Answer:

Time complexity measures how an algorithm's runtime grows as input size increases (e.g. O(n) for a single loop through n elements). Space complexity measures how much additional memory an algorithm needs relative to input size (e.g. O(1) for an in-place swap vs O(n) for creating a copy of the array). Both are usually expressed using Big-O notation and both matter when choosing an algorithm for constrained environments.

Q63. Compare Merge Sort and Quick Sort - discuss best/worst case complexity, stability, and when you'd choose one over the other.

Answer:

Merge Sort has guaranteed O(n log n) time in best, average, and worst cases, is stable, but requires O(n) extra space for merging. Quick Sort also averages O(n log n) but can degrade to O(n^2) in the worst case (e.g. already-sorted input with a poor pivot choice), is typically not stable, but sorts in-place with better real-world constant factors and cache performance. Choose Merge Sort when stability or guaranteed worst-case performance matters (e.g. external sorting); choose Quick Sort for general in-memory sorting where average-case speed matters more.

Sorting Algorithm Complexity Comparison

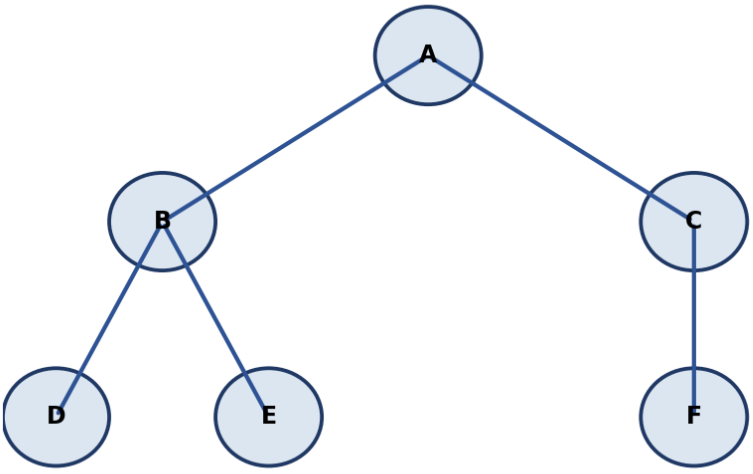
Algorithm	Best	Average	Worst	Stable?
Bubble Sort	O(n)	O(n^2)	O(n^2)	Yes
Selection Sort	O(n^2)	O(n^2)	O(n^2)	No
Insertion Sort	O(n)	O(n^2)	O(n^2)	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	Yes
Quick Sort	O(n log n)	O(n log n)	O(n^2)	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	No

Q64. Explain the difference between BFS and DFS graph traversal, including their use cases.

Answer:

BFS (Breadth-First Search) explores a graph level by level using a queue, visiting all neighbors of a node before moving deeper - useful for finding the shortest path in an unweighted graph. DFS (Depth-First Search) explores as far as possible along one branch before backtracking, using a stack (or recursion) - useful for tasks like detecting cycles, topological sorting, or exploring all possible paths/states.

Graph: BFS order A,B,C,D,E,F | DFS order A,B,D,E,C,F



Q65. Explain Dijkstra's algorithm for shortest paths and its limitations with negative edge weights.

Answer:

Dijkstra's algorithm finds the shortest path from a source node to all others in a weighted graph by repeatedly selecting the unvisited node with the smallest known distance, then 'relaxing' (updating) the distances of its neighbors - using a priority queue/min-heap, it runs in O(E log V). Its key limitation is that it assumes all edge weights are non-negative; a negative edge can invalidate its greedy assumption that once a node is finalized its distance can't improve, requiring an algorithm like Bellman-Ford instead.

Q66. Explain how binary search works and derive its time complexity.

Answer:

Binary search repeatedly divides a sorted array in half: compare the target to the middle element, and discard the half that can't contain the target, narrowing the search range each time. Since the search space is halved every step, the time complexity is O(log n) - a huge improvement over linear search's O(n) for large sorted datasets.

Q67. What is Big-O notation, and why do we use asymptotic analysis instead of exact runtimes?

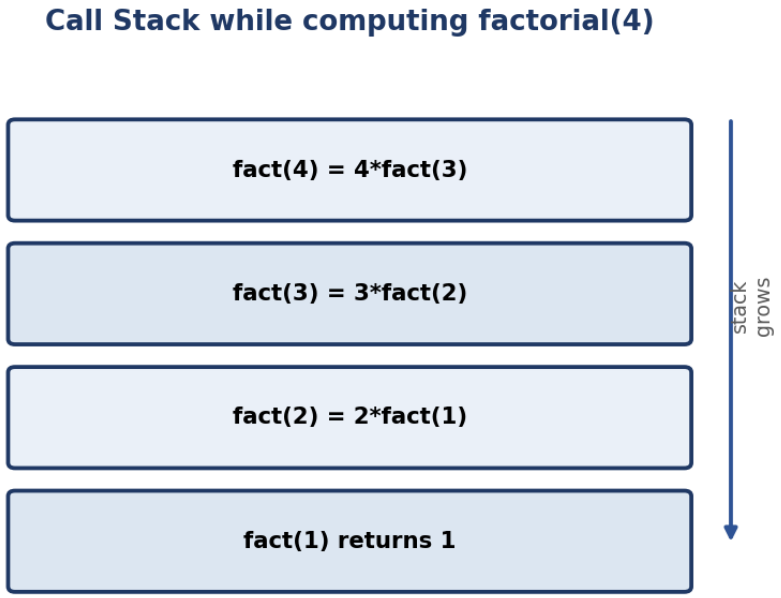
Answer:

Big-O notation describes the upper-bound growth rate of an algorithm's running time or space as input size approaches infinity, ignoring constant factors and lower-order terms. We use asymptotic analysis instead of exact runtimes because exact timings vary by hardware, language, and implementation details, while Big-O gives a hardware-independent way to compare how algorithms scale as input grows large.

Q68. What is dynamic programming, and how does it differ from plain recursion and from greedy algorithms?

Answer:

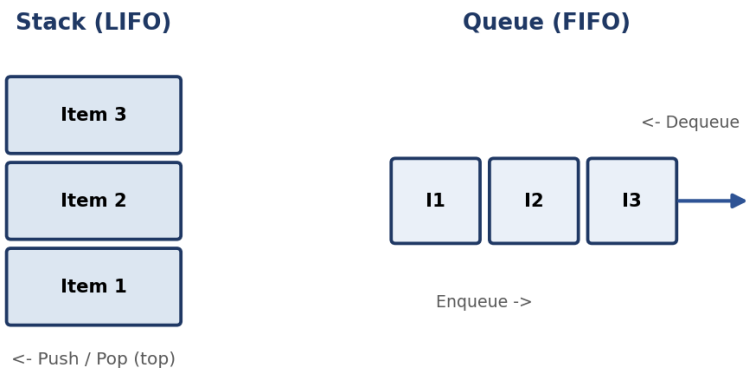
Dynamic programming solves problems by breaking them into overlapping subproblems, solving each just once, and storing (memoizing) the results to avoid redundant recomputation - it's applicable when a problem has both optimal substructure and overlapping subproblems. Plain recursion without memoization re-solves the same subproblems repeatedly, often leading to exponential time. Unlike a greedy algorithm (which makes one irrevocable locally-optimal choice at each step), DP considers combinations of subproblem solutions to guarantee a globally optimal result when applicable.



Q69. What is the difference between a stack and a queue, and where is each used in real systems?

Answer:

A stack follows Last-In-First-Out (LIFO) order - used for undo functionality, expression evaluation, and the call stack behind recursion. A queue follows First-In-First-Out (FIFO) order - used for task scheduling, breadth-first search, and handling requests in the order they arrive (e.g. print queues, message queues).



Q70. How would you merge two sorted arrays/linked lists efficiently?

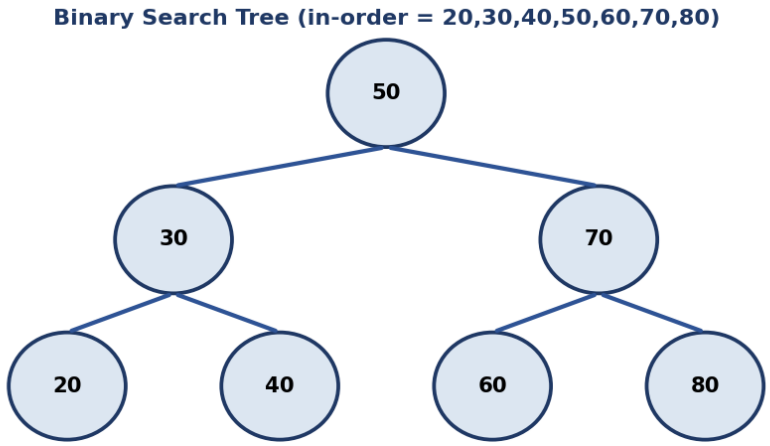
Answer:

Use a two-pointer merge: maintain one pointer into each sorted array/list, repeatedly compare the elements they point to, append the smaller one to the result, and advance that pointer - continue until one input is exhausted, then append the remainder of the other. This runs in O(n + m) time, which is also the core merge step used inside Merge Sort.

Q71. What is a binary search tree, and what is the time complexity of insertion, deletion, and search in the average and worst case?

Answer:

A Binary Search Tree (BST) is a tree where every node's left subtree contains only smaller values and its right subtree contains only larger values, enabling efficient ordered operations. In a balanced BST, insertion, deletion, and search are all O(log h) where h is the height (roughly log n for a balanced tree). In the worst case (a degenerate, list-like tree caused by inserting sorted data without rebalancing), all three operations degrade to O(n).



Q72. Explain amortized time complexity using the example of a dynamic array's append operation.

Answer:

Amortized time complexity averages the cost of an operation over a sequence of operations, even if occasional individual operations are expensive. A dynamic array's append is usually O(1), but when the underlying fixed-size array is full, it must allocate a new (typically double-sized) array and copy all existing elements - an O(n) operation. Because this expensive resize happens increasingly rarely as the array grows, the average cost per append across many operations still works out to O(1) amortized.

Coding Problems

Q73. Write a program to merge overlapping intervals given a list of intervals.

Answer:

Sort the intervals by their start time, then iterate through them while maintaining a 'current merged interval' - if the next interval's start is less than or equal to the current merged interval's end, merge them by extending the end to the maximum of the two ends; otherwise, push the current merged interval to the result and start a new one with the next interval. This runs in O(n log n) time due to the initial sort.

Q74. Write a program to remove duplicates from a sorted array in-place.

Answer:

Since the array is already sorted, use a slow pointer marking the last position of a unique element written so far, and a fast pointer scanning ahead - whenever the fast pointer finds a value different from the one at the slow pointer, advance the slow pointer and copy that new value into place. This removes duplicates in-place in O(n) time and O(1) extra space.

Q75. Write a program to check if two strings are anagrams of each other.

Answer:

Two common approaches: (1) sort both strings and check if the sorted versions are equal (O(n log n)); or (2) build a frequency count (e.g. a 26-length array for lowercase letters, or a hashmap for general characters) of one string, then decrement counts while scanning the second string - if all counts return to zero and lengths match, they're anagrams (O(n) time, O(1) or O(k) space).

Q76. Write a program to find the maximum subarray sum (Kadane's Algorithm).

Answer:

Kadane's Algorithm scans the array once, maintaining a running 'current sum' that resets to the current element whenever adding the previous running sum would make it smaller than the element alone (i.e. current_sum = max(arr[i], current_sum + arr[i])), while tracking the overall maximum seen so far. This finds the maximum subarray sum in O(n) time and O(1) space.

Q77. Write a program to find the intersection of two arrays.

Answer:

Efficient approach: put all elements of the smaller array into a hash set for O(1) lookup, then iterate through the other array and collect any element found in the set (adding it to a result set to avoid duplicates in the output). This runs in O(n + m) time and O(min(n, m)) space.

Q78. Write a program to check whether a given string is a palindrome.

Answer:

Use two pointers, one from the start (left) and one from the end (right) of the string, comparing characters and moving both pointers inward - if any pair doesn't match, it's not a palindrome; if the pointers meet or cross without mismatches, it is. This runs in O(n) time and O(1) extra space (ignoring the input itself).

Q79. Write a program to find the missing number in an array containing n distinct numbers from 0 to n.

Answer:

Sum formula approach: the expected sum of numbers 0 to n is n*(n+1)/2; subtract the actual sum of the given array from this expected sum, and the difference is the missing number - O(n) time, O(1) space. Alternatively, XOR every number from 0 to n together with every element in the array; since XOR-ing a number with itself cancels out to zero, all present numbers cancel, leaving only the missing number.

Q80. Write a program to check whether a binary tree is balanced.

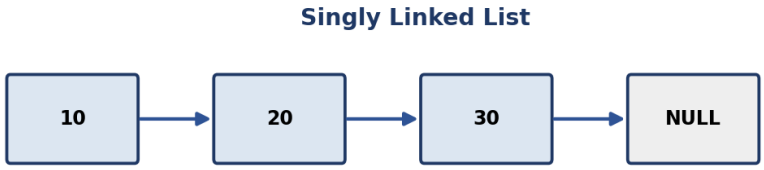
Answer:

Recursively compute the height of the left and right subtrees for every node; if at any point the difference in height between left and right subtrees exceeds 1 for any node, the tree is unbalanced. An efficient implementation computes height and checks balance in the same bottom-up traversal (returning -1 as a sentinel to signal 'unbalanced' early), achieving O(n) time instead of the naive O(n^2).

Q81. Write a program to reverse a singly linked list (iterative and recursive approaches).

Answer:

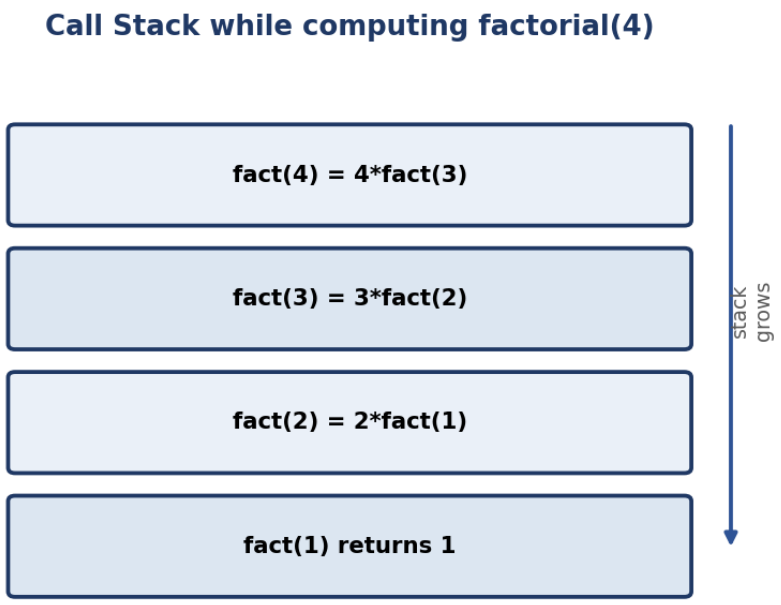
Iterative: maintain a 'previous' pointer (initially null) and a 'current' pointer (initially head); at each step, save current.next, point current.next to previous, then advance previous and current forward - repeat until current is null, then previous is the new head. Recursive: recurse to the end of the list first, then reverse the pointer of each node on the way back up (node.next.next = node; node.next = null). Both run in O(n) time; iterative uses O(1) extra space while recursive uses O(n) call-stack space.



Q82. Write a program to find the factorial of a number using recursion.

Answer:

Recursive definition: factorial(n) = n * factorial(n-1), with a base case factorial(0) = 1 (or factorial(1) = 1). Each call multiplies n by the result of the smaller subproblem until the base case is reached, then the multiplications unwind back up the call stack. Time complexity is O(n); space complexity is O(n) due to the recursive call stack.



Q83. Write a program to find the first non-repeating character in a string.

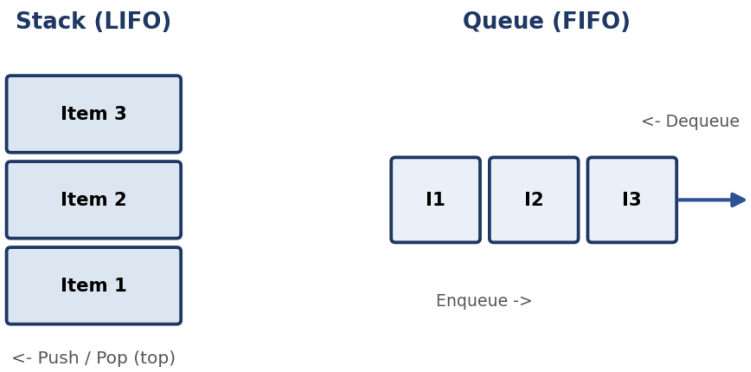
Answer:

Use a hashmap to count the frequency of each character in a single pass through the string, then iterate through the string again in order and return the first character whose count is exactly 1. This runs in O(n) time and O(k) space, where k is the size of the character set.

Q84. Write a program to implement a stack using two queues (or vice versa).

Answer:

To implement a queue using two stacks: push new elements onto stack1; for dequeue, if stack2 is empty, pop everything from stack1 and push it onto stack2 (reversing the order), then pop from stack2 - this gives FIFO behavior with amortized O(1) per operation. Implementing a stack using two queues works similarly in reverse, rearranging elements between the two queues so the most recently added element comes out first.



Q85. Write a program to implement binary search on a rotated sorted array.

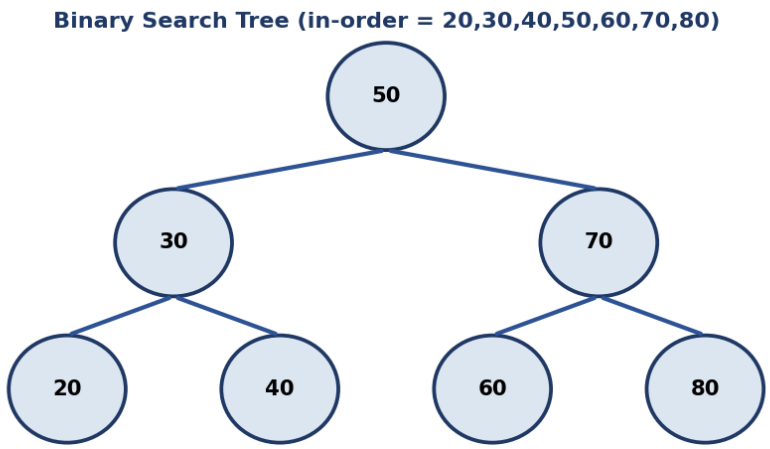
Answer:

Modify binary search by first determining which half of the array (from mid to low, or mid to high) is properly sorted, then checking if the target lies within that sorted half's range - if so, search recurse/iterate into that half, otherwise search the other half. This preserves the O(log n) time complexity of standard binary search despite the rotation.

Q86. Write a program to find the lowest common ancestor (LCA) of two nodes in a binary tree.

Answer:

For a BST, the LCA can be found by starting at the root and comparing both target node values to the current node's value: if both are smaller, move to the left child; if both are larger, move to the right child; if they diverge (one smaller, one larger, or one equals the current node), the current node is the LCA. This runs in O(h) time where h is the tree height. For a general binary tree (not necessarily a BST), a recursive post-order search that returns the first node where both targets are found in different subtrees is used instead.



Programming Language Fundamentals

Q87. What is exception handling, and why is it important to avoid empty catch blocks?

Answer:

Exception handling is the mechanism for detecting and responding to runtime errors (like a missing file or invalid input) gracefully, using constructs like try/catch/finally, instead of letting the program crash. Empty catch blocks are dangerous because they silently swallow errors - the program continues running as if nothing went wrong, making it far harder to diagnose bugs later since there's no log, message, or fallback behavior indicating that something actually failed.

Q88. What is a memory leak, and how would you identify one in a long-running application?

Answer:

A memory leak occurs when a program allocates memory but never releases it even though it's no longer needed, causing memory usage to grow over time and potentially exhausting available memory. In a long-running application, you'd identify one using a memory profiler to take heap snapshots over time and look for object counts/memory usage that keep growing without bound, then trace those objects back to what's still holding references to them (e.g. an ever-growing cache or a forgotten event-listener registration).

Q89. Explain the difference between checked and unchecked exceptions in Java.

Answer:

Checked exceptions (e.g. IOException) are checked by the compiler at compile time - a method must either handle them with try-catch or declare them with 'throws', forcing the caller to acknowledge the possibility of failure. Unchecked exceptions (e.g. NullPointerException, extending RuntimeException) are not checked at compile time and don't require explicit handling, typically representing programming errors that could occur almost anywhere.

Q90. What is the difference between '==' and '.equals()' when comparing objects in Java?

Answer:

'==' compares object references - whether two variables point to the exact same object in memory. '.equals()' compares the logical content/value of two objects, based on however the class has overridden it (e.g. String's .equals() compares character sequences). Two distinct String objects with the same text will be '==' false but '.equals()' true, unless string interning makes them the same reference.

Q91. What are lambda expressions/anonymous functions, and when would you use them?

Answer:

A lambda expression (or anonymous function) is a small, unnamed function defined inline, typically used for short operations passed as arguments to other functions - e.g. sorting a list with a custom key: sorted(items, key=lambda x: x.age). They're best used for simple, one-off logic; anything more complex or reused in multiple places is usually clearer as a named function.

Q92. Explain Python's Global Interpreter Lock (GIL) and its impact on multithreaded CPU-bound programs.

Answer:

CPython's Global Interpreter Lock ensures only one thread executes Python bytecode at any given instant, even on a multi-core machine, to simplify memory management (reference counting) and avoid certain race conditions internally. This means CPU-bound multithreaded Python code doesn't actually get faster from adding more threads (they contend for the GIL) - for true parallel CPU-bound work, developers typically use multiprocessing (separate processes, each with its own interpreter and GIL) instead.

Q93. Explain the difference between a shallow copy and a deep copy of a nested data structure.

Answer:

A shallow copy creates a new outer object but copies references to the same nested objects as the original - mutating a nested list inside a shallow copy also affects the original. A deep copy recursively creates independent copies of every nested object, so the copy is fully isolated from the original at every level, at the cost of more time and memory to perform the copy.

Q94. What is duck typing, and how does it relate to Python's dynamic type system?

Answer:

Duck typing means an object's suitability for an operation is determined by whether it has the necessary methods/attributes ('if it walks like a duck and quacks like a duck...') rather than its explicit declared type. Python's dynamic typing supports this naturally - a function can accept any object as long as it implements the methods the function actually calls, without requiring a shared base class or interface.

Q95. Explain the difference between an abstract class and a concrete class with an example from a language you know well.

Answer:

An abstract class can declare abstract methods (no body, must be implemented by subclasses) alongside concrete methods with actual implementations, and cannot be instantiated directly - e.g. an abstract Shape class with an abstract area() method but a concrete describe() method common to all shapes. A concrete class provides full implementations for all its methods and can be instantiated directly, e.g. a Circle class extending Shape and implementing area().

Q96. Explain the concept of multithreading vs multiprocessing, and when you would choose one over the other.

Answer:

Multithreading runs multiple threads within a single process, sharing memory - lightweight and fast to communicate between threads, but requires careful synchronization to avoid race conditions, and (in languages like Python with a GIL) doesn't achieve true CPU parallelism. Multiprocessing runs separate processes, each with its own memory space - avoids shared-memory race conditions and achieves true parallelism across CPU cores, but has higher overhead for creation and inter-process communication. Choose multithreading for I/O-bound work and multiprocessing for CPU-bound work in GIL-constrained languages.

Web & Cloud Fundamentals

Q97. Explain the difference between IaaS, PaaS, and SaaS with real examples of each.

Answer:

IaaS (Infrastructure as a Service) provides raw virtualized compute/storage/networking that you configure yourself, e.g. AWS EC2 or Azure VMs. PaaS (Platform as a Service) provides a managed platform to build and deploy applications without managing the underlying servers, e.g. Heroku or Azure App Service. SaaS (Software as a Service) delivers a complete, ready-to-use application over the internet, e.g. Gmail or Salesforce - you just use it, with no infrastructure or platform management at all.

Q98. What is a RESTful API, and what makes an API design 'RESTful'?

Answer:

A RESTful API is an API designed around REST (Representational State Transfer) architectural principles: it uses standard HTTP methods (GET, POST, PUT, DELETE) mapped to resource operations, is stateless (each request contains all information needed), uses resource-based URLs (e.g. /users/123), and typically returns representations of resources (often JSON). An API is 'RESTful' to the extent it actually follows these constraints rather than just using HTTP as a transport.

Q99. What is containerization (e.g., Docker), and how is it different from a virtual machine?

Answer:

Containerization packages an application with its dependencies and runtime into a lightweight, portable container (e.g. via Docker) that shares the host OS kernel, making containers fast to start and resource-efficient. A virtual machine, by contrast, virtualizes an entire hardware stack including its own full guest OS, making VMs heavier and slower to start but providing stronger isolation between workloads.

Q100. Explain how a message queue like Kafka or RabbitMQ helps decouple microservices.

Answer:

A message queue like Kafka or RabbitMQ sits between producer services (that generate events/messages) and consumer services (that process them), letting producers publish messages without waiting for consumers to be ready or fast. This decouples services in time and load - producers and consumers can scale, fail, or be deployed independently - and provides buffering so a burst of traffic doesn't overwhelm downstream consumers directly.

Note: 20 of these 100 questions include a supporting diagram. Content compiled from publicly available 2026 placement-prep resources and established CS-fundamentals question banks - verify current-year specifics before your interview. Practice explaining answers out loud or in writing, not just reading - recall under pressure is the real test.